

ROGIS

Documento de Planeación Estratégica y Técnica

Un cache-database diseñado para tráfico de agentes de IA, no de humanos — superset determinístico de Redis + capa semántica nativa.

Preparado como documento base antes de escribir código. Objetivo: dejar cerradas las decisiones de producto, arquitectura y metodología de benchmarking, de forma que la implementación en Rust sea un proceso de ejecución sobre un plan ya validado, no de descubrimiento.

Proyecto 1 de una serie de experimentos: construir productos técnicamente ambiciosos guiándose por conversación con IA, sin experiencia previa en el dominio.

00 Resumen ejecutivo

Redis fue diseñado en 2009 para un patrón de tráfico muy específico: aplicaciones web con humanos detrás, haciendo peticiones puntuales, estructuradas y de baja redundancia semántica. Ese patrón ya no describe a la mayoría del tráfico nuevo que se está construyendo en 2026: agentes de IA que iteran, reintentan, exploran múltiples caminos para la misma tarea, y generan consultas en lenguaje natural en vez de claves estructuradas.

Rogis parte de una tesis simple: existe espacio para un cache-database pensado desde cero para ese patrón de acceso — alto volumen de redundancia, consultas difusas en vez de exactas, costo de miss mucho más alto que en un stack tradicional (un miss puede disparar una llamada a LLM de varios segundos y centavos de dólar, no solo una query SQL de milisegundos).

La estrategia de producto tiene dos capas no negociables entre sí:

- › **Capa determinista (paridad con Redis):** Rogis debe ser un reemplazo directo (drop-in) de Redis para los usos que un agente de IA también necesita — estado de sesión, rate limiting, colas, locks, pub/sub. Si esto falla, cualquier adoptante necesita Redis + Rogis, y el proyecto pierde su razón de existir.
- › **Capa semántica (el diferenciador real):** almacenamiento nativo de vectores/embeddings dentro del motor — sin pasar por serialización genérica de bytes — con búsqueda por similitud integrada como primitiva de primera clase, no como una librería wrapper por encima.

Este documento no contiene código. Es el artefacto que debe existir *antes* del código: mapa competitivo, decisiones de arquitectura pendientes, especificación funcional de comandos, diseño de benchmarks, y — sección más importante — la lista de preguntas abiertas que hay que responder con evidencia (no con intuición) antes de escribir la primera línea de Rust.

Cómo leer este documento: cada sección técnica termina con preguntas explícitas marcadas en cursiva. Esas preguntas son literalmente el contenido de la próxima conversación — no se responden solas, se responden decidiendo.

01 Tesis de producto y posicionamiento

Frase de posicionamiento: "Redis fue construido para servir a humanos a través de aplicaciones. Rogis está construido para servir a agentes de IA que actúan por su cuenta."

Esto no es solo marketing — implica decisiones técnicas concretas y verificables:

Dimensión	Tráfico humano (Redis clásico)	Tráfico de agentes (Rogis)
Forma de la consulta	Clave estructurada y exacta (user:123)	Mezcla de claves exactas y lenguaje natural / prompts
Redundancia	Baja — cada request suele ser distinto	Alta — un agente reintentará y explora variaciones de la misma tarea
Costo de un miss	Bajo (una query a DB, ms)	Alto (una llamada a LLM, segundos + \$)
Volumen por 'unidad de trabajo'	1 request ≈ 1 acción de usuario	1 tarea de agente ≈ 10-50 llamadas internas
Formato de salida esperado	JSON/HTML para renderizar en UI	Texto compacto listo para meter en el context window
Tolerancia a coordinación concurrente	Locks ocasionales	Múltiples agentes/sub-agentes operando en paralelo sobre el mismo estado

Qué NO es Rogis

Delimitar el alcance es tan importante como definirlo. Rogis explícitamente no busca ser:

- › Una base de datos vectorial de propósito general para RAG a gran escala (eso es terreno de Pinecone, Weaviate, Qdrant — bases pensadas para millones de documentos persistentes, no para cache efímero).
- › Una base de datos multi-modelo todo-en-uno tipo SurrealDB — Rogis se queda deliberadamente estrecho: es un cache, no un almacén de verdad (source of truth).
- › Un framework de orquestación de agentes — Rogis es infraestructura de datos, no lógica de agente.

Preguntas abiertas de esta sección:

- › ¿El público objetivo inicial son builders de agentes individuales (indie hackers) o equipos de startups con producto en producción? Esto cambia el nivel de pulido / documentación necesario para el MVP.
- › ¿Nos posicionamos primero como 'Redis para agentes' (paraguas amplio) o como 'semantic cache que también hace todo lo de Redis' (empezar angosto y expandir)? El orden de construcción depende de esto.

02 Panorama competitivo

Mapeo de quién ocupa qué parte del espacio, basado en investigación reciente (2026). El objetivo de esta sección es evitar construir algo que ya existe con otro nombre.

Proyecto	Qué es	Dónde compete	Gap que deja abierto
Redis / Valkey	Cache KV clásico en memoria	Determinismo, velocidad de hash lookup, ecosistema masivo	No entiende embeddings de forma nativa; todo es bytes serializados
Upstash Semantic Cache	Librería sobre Upstash Vector (cloud, no Redis)	Zero-ops, serverless, integración rápida	Depende de red (round-trip a la nube); no hay despliegue local
BetterDB	Librería cliente sobre Valkey/Redis existente	Observabilidad, tracking de \$ ahorrados, auto-tuning de threshold	Sigue serializando vectores como bytes genéricos dentro de Redis — no cambia el motor
GPTCache / RedisVL	Librerías Python de semantic cache sobre Redis	Integración con frameworks RAG (LangChain, LlamaIndex)	Misma limitación: capa de aplicación, no motor de almacenamiento nuevo
Pinecone / Qdrant / Weaviate	Bases de datos vectoriales persistentes	Escala masiva de vectores para retrieval / RAG	No son caches (no están pensadas para TTL, eviction, ni baja latencia efímera)
SurrealDB	Base de datos multi-modelo (docs, grafos, vectores, series de tiempo)	Consolidar varias DBs en una sola para agentes con memoria persistente	Es una base de datos de verdad (source of truth), no un cache — otra categoría

Conclusión del mapeo: el hueco que nadie ocupa hoy es exactamente la intersección de tres cosas — (a) motor propio, no librería sobre Redis existente, (b) vectores como ciudadanos de primera clase en el storage layer (no serializados), y (c) diseño explícito para el patrón de acceso de agentes, no de RAG genérico. Ese es el espacio de Rogis.

Preguntas abiertas de esta sección:

- › *Antes de escribir código: ¿hay que correr los benchmarks públicos de BetterDB/RedisVL nosotros mismos para confirmar que las cifras de esta sección siguen vigentes al momento de construir? (proyectos AI se mueven rápido, esta tabla puede quedar desactualizada en meses).*
- › *¿Existe algún proyecto en Rust específicamente (no Python/TS) haciendo semantic cache a nivel de motor que no hayamos encontrado todavía? Vale la pena una búsqueda dedicada en crates.io y GitHub antes de arrancar.*

03 Arquitectura general — dos capas

Principio rector: **nadie debe necesitar Redis + Rogis a la vez**. Rogis debe ser un superset funcional, no un cache adicional que vive al lado de Redis.

Capa 1 — Paridad determinista

Objetivo: cualquier aplicación que hoy usa Redis para lo básico debe poder apuntar a Rogis sin cambiar código de cliente.

- › **Protocolo:** implementar RESP (REdis Serialization Protocol) para compatibilidad con clientes existentes (redis-py, ioredis, go-redis, etc.) — esto es lo que da adopción 'drop-in' real, no solo una API parecida.
- › **Tipos de datos base:** strings, hashes, listas, sets, sorted sets — los estructurales que cualquier app usa para estado de sesión y datos de agente.
- › **Primitivas de coordinación:** `SETNX`/`locks` distribuidos, `EXPIRE`/`TTL`, `INCR`/`DECR` (rate limiting), `pub/sub` para coordinación entre agentes o entre agente y frontend.

Capa 2 — Motor semántico nativo

Objetivo: el diferenciador real, construido como extensión del mismo motor, no como servicio aparte.

- › **Tipo de dato nuevo:** un tipo "vector" nativo — array de floats de dimensión fija, almacenado sin pasar por serialización de string, alineado en memoria para operaciones SIMD.
- › **Comandos nuevos:** variantes semánticas de SET/GET que aceptan un vector (o calculan uno vía un modelo de embeddings embebido) y buscan por similitud sobre un índice aproximado (ANN).
- › **Índice de similitud:** HNSW (Hierarchical Navigable Small World) es el estándar de facto para ANN en 2026 — hay implementaciones en Rust (`hnsw_rs`, `instant-distance`) que se pueden evaluar antes de decidir escribir uno propio.

Diagrama conceptual de capas

Cliente (redis-py, ioredis, SDK propio de Rogis)
Protocolo RESP + extensión de comandos semánticos
Capa 1 · Motor determinista (hash tables, listas, TTL, pub/sub, locks)
Capa 2 · Motor semántico (vectores nativos + índice HNSW + comparación SIMD)
Storage engine compartido (memoria + persistencia opcional a disco)

El punto crítico de diseño: ambas capas comparten el mismo storage engine de base — no son dos servicios separados unidos por red.

Preguntas abiertas de esta sección:

- › ¿Rogis corre *single-threaded* con *event loop* (como Redis clásico) o *multi-threaded* desde el diseño inicial? Rust permite concurrencia segura que Redis (en C) evita deliberadamente — pero cambia todo el modelo de

consistencia.

› *¿Persistencia a disco desde el MVP (tipo RDB/AOF de Redis) o 100% en memoria al inicio, con persistencia como fase 2? Un cache puro puede vivir sin persistencia; pero si se posiciona como 'reemplazo de Redis', puede que se espere.*

› *¿El modelo de embeddings corre embebido dentro de Rogis (vía ONNX Runtime en Rust) o siempre se recibe el vector ya calculado desde el cliente? Afecta drásticamente la complejidad del MVP.*

04 Especificación funcional — comandos deterministas (Capa 1)

Lista priorizada de comandos a implementar primero, elegidos por uso real en apps de agentes (no la lista completa de 200+ comandos de Redis). El criterio de corte: ¿un agente típico (LangChain/LangGraph/custom loop) necesita esto para funcionar?

Categoría	Comandos	Caso de uso en un agente
Strings / TTL	SET, GET, EXPIRE, TTL, DEL	Cache de resultado de tool-call exacto (mismos parámetros)
Contadores	INCR, DECR, INCRBY	Rate limiting de llamadas a LLM por usuario/sesión
Hashes	HSET, HGET, HGETALL, HDEL	Estado estructurado de sesión / memoria de corto plazo del agente
Listas / Streams	LPUSH, RPOP, XADD, XREAD	Cola de tareas entre agentes; historial de conversación append-only
Sets	SADD, SISMEMBER, SREM	Deduplicación de tareas ya procesadas por un agente
Locks	SETNX, lock con TTL	Evitar que dos sub-agentes procesen la misma tarea a la vez
Pub/Sub	SUBSCRIBE, PUBLISH	Coordinación entre agente y frontend/otros agentes en tiempo real

Criterio de 'hecho' para esta capa: correr un benchmark idéntico (mismo volumen de operaciones) contra Redis/Valkey real usando estos comandos, y no estar significativamente por debajo en throughput ni latencia. No hace falta ganarle a Redis aquí — hace falta no perder.

Preguntas abiertas de esta sección:

- › ¿Streams (XADD/XREAD, usados para historial append-only de conversación) entran en el MVP o se posponen? Son más complejos de implementar bien que listas simples.
- › ¿Vale la pena copiar también TTL con precisión de milisegundo como Redis, o alcanza con segundos para el caso de uso de agentes (que suele ser menos sensible a esa precisión)?

05 Especificación funcional — comandos semánticos (Capa 2, el diferenciador)

Estos son comandos que Redis no tiene y que constituyen la propuesta de valor central. Nombres tentativos, a definir mejor durante el diseño de la API.

Comando (tentativo)	Función	Parámetros clave
SEMSET	Guarda un par (texto o vector) → valor, indexado por embedding	clave lógica, texto o vector, valor, TTL, namespace/colección
SEMGET	Busca por similitud semántica contra lo almacenado	texto o vector de consulta, threshold de similitud, top-k
SEMSTATS	Métricas del cache semántico: hit rate, tamaño del índice, distribución de scores	namespace opcional
SEMTUNE	Ajuste asistido del threshold en base al tráfico reciente (inspirado en auto-tuning)	ventana de tiempo, modo (sugerir vs aplicar automático)

Decisiones de diseño pendientes para esta capa

- › **Umbral de similitud por defecto:** según la evidencia de mercado, el threshold óptimo no es portable entre runtimes de embeddings distintos — depende del modelo usado y la distribución real de scores. Rogis debería exponer una forma de medir esto empíricamente en vez de sugerir un número fijo en la documentación.
- › **Multi-tenancy / namespaces:** distintas aplicaciones (o distintos tipos de contenido dentro de la misma app) probablemente necesitan índices semánticos separados, con su propio threshold.
- › **Qué pasa en un miss:** ¿Rogis solo informa 'no hit' o puede orquestar la llamada al proveedor de embeddings / LLM directamente? Empezar con lo primero (más simple, menos acoplamiento) es más seguro para el MVP.

Preguntas abiertas de esta sección:

- › ¿El formato de salida de SEMGET debe poder pedirse ya 'listo para prompt' (texto plano compacto) en vez de JSON, aprovechando el eje de 'cache para agentes, no para humanos'?
- › ¿Compatibilidad con proveedores de embeddings externos (OpenAI, Voyage, Cohere) desde el día uno, o el MVP fuerza un modelo local embebido primero y se abre después?
- › ¿SEMSET debe soportar actualizar/invalidar una entrada cuando el dato de origen cambia (invalidación por contenido), o el TTL simple alcanza para la primera versión?

06 Decisiones técnicas de implementación en Rust

Roger viene del mundo AI/producto, no de sistemas de bajo nivel — esta sección existe para dejar explícitas las decisiones que normalmente se toman "por experiencia" y que aquí hay que tomar de forma consciente, investigando cada una antes de fijarla.

Sobre copiar el repo de Redis como punto de partida

La idea de partir del repo de Redis tiene un matiz importante: el Redis original está escrito en C, no en Rust — no se puede "copiar" literalmente el código y extenderlo en Rust. Lo que sí es valioso copiar/estudiar de Redis:

- › La **especificación del protocolo RESP** (esto sí es texto/spec, no código C — se puede implementar en Rust directamente desde la documentación oficial del protocolo).
- › El **conjunto de comandos y su semántica exacta** (qué devuelve GET sobre una clave inexistente, comportamiento de EXPIRE sobre distintos tipos, etc.) — esto es lo que garantiza compatibilidad real.
- › La **suite de tests de compatibilidad** de Redis (si es pública/reutilizable) como criterio objetivo de "funciona igual que Redis", en vez de confiar en intuición.

Lo que NO tiene sentido copiar: la implementación en C línea por línea — Rust tiene primitivas de concurrencia y manejo de memoria completamente distintas (ownership, sin garbage collector) que hacen que una traducción literal sea, en el mejor caso, subóptima.

Decisiones de motor a resolver antes de codear

Decisión	Opciones	Impacto
Estructura del store base	HashMap propio vs. <code>crate `dashmap`</code> (concurrente)	Afecta throughput bajo concurrencia real desde el primer benchmark
Modelo de concurrencia	Single-thread + event loop (tipo Redis) vs. multi-thread nativo	Cambia el modelo de consistencia y la complejidad de locks internos
Índice ANN	Crate existente (<code>`hnsw_rs`</code> , <code>`instant-distance`</code>) vs. implementación propia	Empezar con un crate maduro reduce riesgo; escribir uno propio es una historia aparte, no bloqueante para el MVP
Runtime async	<code>`tokio`</code> (estándar de facto) vs. I/O bloqueante simple	<code>`tokio`</code> es casi obligatorio para manejar muchas conexiones concurrentes de forma eficiente
Embeddings embebidos	ONNX Runtime en Rust (<code>`ort`</code>) vs. delegar siempre al cliente	Embeddings embebidos evitan dependencias externas pero añaden peso y complejidad al binario
Persistencia	Snapshot periódico a disco vs. 100% en memoria (MVP)	Determina si Rogis puede usarse en producción real desde el día uno o solo como cache descartable

Preguntas abiertas de esta sección:

- › ¿Cuál de estas decisiones se toma por investigación previa (leyendo benchmarks/documentación de cada crate) y cuál se toma "probando las dos" en un prototipo descartable de unos días?
- › ¿El objetivo del MVP es correr en la misma máquina que la app (embebido / co-localizado, aprovechando shared memory) o como servicio de red independiente igual que Redis? Esto es una decisión de arquitectura

mayor que afecta todo lo demás — vale la pena resolverla primero, no en paralelo.

07 Estrategia de benchmarking para casos de uso de IA

El posicionamiento de Rogis depende de números defendibles, no de afirmaciones. Esta sección diseña los benchmarks antes de tener el producto, para que la implementación se construya ya sabiendo qué se va a medir.

Principio: separar lo que es arquitectónico de lo que es del motor

La investigación de mercado mostró un error común en benchmarks de la competencia: medir "cache local vs. API en la nube" y presentarlo como si fuera una ventaja del motor. Rogis debe evitar ese mismo error — todo benchmark debe dejar explícito qué variable se está aislando.

Benchmark	Qué mide	Contra qué se compara
Paridad determinista	Throughput y latencia en SET/GET/HSET/INCR con volumen alto y concurrencia	Redis / Valkey real, misma máquina, mismo protocolo RESP
Storage nativo de vectores	Latencia de SET/GET de embeddings, con y sin serialización	Redis almacenando el mismo vector como string serializado (JSON o binario)
Semantic cache — calidad	Precision/Recall (F1) del hit/miss frente a pares etiquetados como similares o no	BetterDB y Upstash Semantic Cache sobre datasets públicos (STSb, SICK, PAWS-Wiki)
Semantic cache — latencia end-to-end	Tiempo total: lookup + (si hit) retorno vs. (si miss) llamada real a LLM	Escenario "sin cache" (siempre llamar al LLM) como línea base de referencia
Patrón de agente real	Hit rate en una simulación de agente que reintenta/explora variaciones de la misma tarea	Mismo escenario corriendo sobre Redis + una librería semantic-cache externa (ej. BetterDB)
Costo económico	\$ ahorrados en llamadas a LLM evitadas, sobre un dataset de tráfico simulado de agente	Costo sin ningún cache semántico, usando precios reales de proveedores de LLM

Datasets a usar

- › **Datasets públicos de similitud semántica** (STSb, SICK, PAWS-Wiki) para medir calidad de forma comparable con lo que ya reportó la competencia — permite decir "a paridad de calidad, superamos en X".
- › **Dataset propio de tráfico de agente**: hay que construir (o encontrar) un dataset que simule específicamente reintentos y variaciones de un agente en loop — los datasets genéricos de similitud de oraciones no capturan bien ese patrón, que es el caso de uso central de Rogis.
- › **Escenario adversarial** (tipo PAWS-Wiki): frases que comparten casi todas las palabras pero significan cosas distintas — cualquier cache semántico basado en distancia coseno falla aquí por diseño; hay que medirlo y ser honesto sobre esa limitación, no esconderla.

Preguntas abiertas de esta sección:

- › ¿Quién construye el dataset de "tráfico realista de agente"? ¿Se genera sintéticamente usando un LLM para simular reintentos, o se captura de un agente real corriendo tareas de prueba?
- › ¿Qué modelo(s) de embeddings se usan para los benchmarks — el mismo en todos los competidores comparados, para que la comparación sea justa? (la investigación de mercado mostró que el mismo nombre

de modelo puede dar distribuciones de score distintas según el runtime).

› *¿Se publican los benchmarks con harness reproducible desde el día uno (como hace la competencia), para que la credibilidad del open source dependa de evidencia verificable y no de afirmaciones?*

08 Riesgos y supuestos a validar

Lista honesta de los puntos donde el plan puede estar equivocado. Validar estos supuestos temprano es más barato que descubrirlos después de escribir miles de líneas de Rust.

Riesgo / supuesto	Por qué importa	Cómo validarlo antes de codear
El ahorro de latencia de "motor nativo" vs. "librería sobre Redis" es real y medible	Es la tesis técnica central del diferenciador — si la ganancia es marginal, el pitch se debilita	Prototipo mínimo: comparar SET/GET de un vector de 768 dims serializado en Redis vs. un array nativo en Rust, antes de construir el resto
Existe demanda real de "un solo cache para todo" en vez de Redis + librería semántica encima	Si a los equipos no les molesta tener dos piezas, el argumento de 'superset' pierde fuerza	Conversar con 3-5 builders de agentes reales sobre su stack actual antes de fijar el pitch final
El ecosistema Rust tiene crates suficientemente maduros para RESP, HNSW y ONNX sin reinventar todo	Determina cuánto del tiempo se va en "plomería" vs. en el diferenciador real	Spike técnico de 2-3 días evaluando los 3-4 crates candidatos de cada categoría antes de comprometerse
Un desarrollador solo (autodidacta) puede llevar esto a MVP funcional en un plazo razonable	Afecta el scope del MVP — mejor un MVP angosto que funciona a uno ambicioso que no se termina	Fijar un scope explícitamente reducido para v0.1 (ver sección 9) y tratarlo como línea dura, no aspiracional
El nombre 'Rogis' y el posicionamiento 'post-AI Redis' no chocan con un proyecto existente	Riesgo de marca / confusión si ya existe algo con nombre o pitch muy similar	Búsqueda dedicada de nombre + pitch antes de publicar el repo open source

Preguntas abiertas de esta sección:

› ¿Cuál de estos cinco riesgos, si resultara falso, mataría el proyecto por completo (vs. cuáles solo obligarían a ajustar el pitch)? Vale la pena ordenarlos por severidad antes de empezar a validar.

09 Scope del MVP y roadmap

Definición dura de "MVP mostrable"

Basado en todo lo anterior, la propuesta de scope mínimo para la primera versión pública / demo:

- › Servidor en Rust que habla RESP y soporta el subconjunto de comandos deterministas de la sección 4 (strings, TTL, hashes, contadores, locks básicos — streams y pub/sub avanzado quedan para después).
- › Comandos SEMSET / SEMGET funcionando sobre un índice HNSW de un crate existente, aceptando vectores ya calculados por el cliente (sin embeddings embebidos todavía).
- › Un benchmark reproducible y publicado (no solo afirmado) mostrando: (a) paridad razonable en comandos deterministas vs. Redis, y (b) la ganancia de latencia del storage nativo de vectores vs. guardar el mismo vector serializado en Redis.
- › Documentación mínima + repo open source con licencia clara desde el día uno.

Explícitamente fuera del MVP (fase 2 en adelante)

- › Persistencia a disco / recuperación ante caídas.
- › Embeddings calculados internamente (ONNX embebido).
- › Auto-tuning de threshold (SEMTUNE) — empezar con threshold configurable manualmente.
- › Clustering / alta disponibilidad multi-nodo.
- › SDKs oficiales más allá de compatibilidad RESP básica.

Roadmap sugerido en fases

Fase	Contenido	Criterio de salida
0 — Spikes técnicos	Evaluar crates (RESP, HNSW, dashmap, tokio); prototipo descartable del storage nativo de vectores	Decisiones de la sección 6 cerradas con evidencia, no con intuición
1 — Capa determinista	Implementar comandos de la sección 4 + benchmark vs. Redis	No estar significativamente por debajo de Redis en el benchmark de paridad
2 — Capa semántica	SEMSET/SEMGET + índice HNSW + benchmark de calidad y latencia	Benchmark reproducible mostrando la ganancia frente a Redis+librería externa
3 — Publicación	Repo open source, documentación, benchmark publicado, primeros usuarios piloto	Al menos un equipo externo probándolo en un proyecto real, no solo interés pasivo

Preguntas abiertas de esta sección — las últimas antes de codear:

- › ¿La Fase 0 (spikes) tiene un límite de tiempo fijo antes de forzar una decisión, para no quedarse evaluando crates indefinidamente?
- › ¿Qué significa concretamente "benchmark reproducible y publicado" — un script en el repo que cualquiera pueda correr, o un reporte tipo blog post como el de la competencia analizada en la sección 2?
- › ¿Cuál es la definición de "listo para mostrar a otros" — un repo con README, o una demo interactiva (ej. un playground web) donde alguien pueda probar Rogis sin instalar nada?

Siguiente paso inmediato sugerido: resolver las preguntas de la Fase 0 (sección 6 y 9) en la próxima conversación — son las únicas que bloquean literalmente escribir la primera línea de código. El resto del documento puede seguir refinándose en paralelo mientras el spike técnico corre.